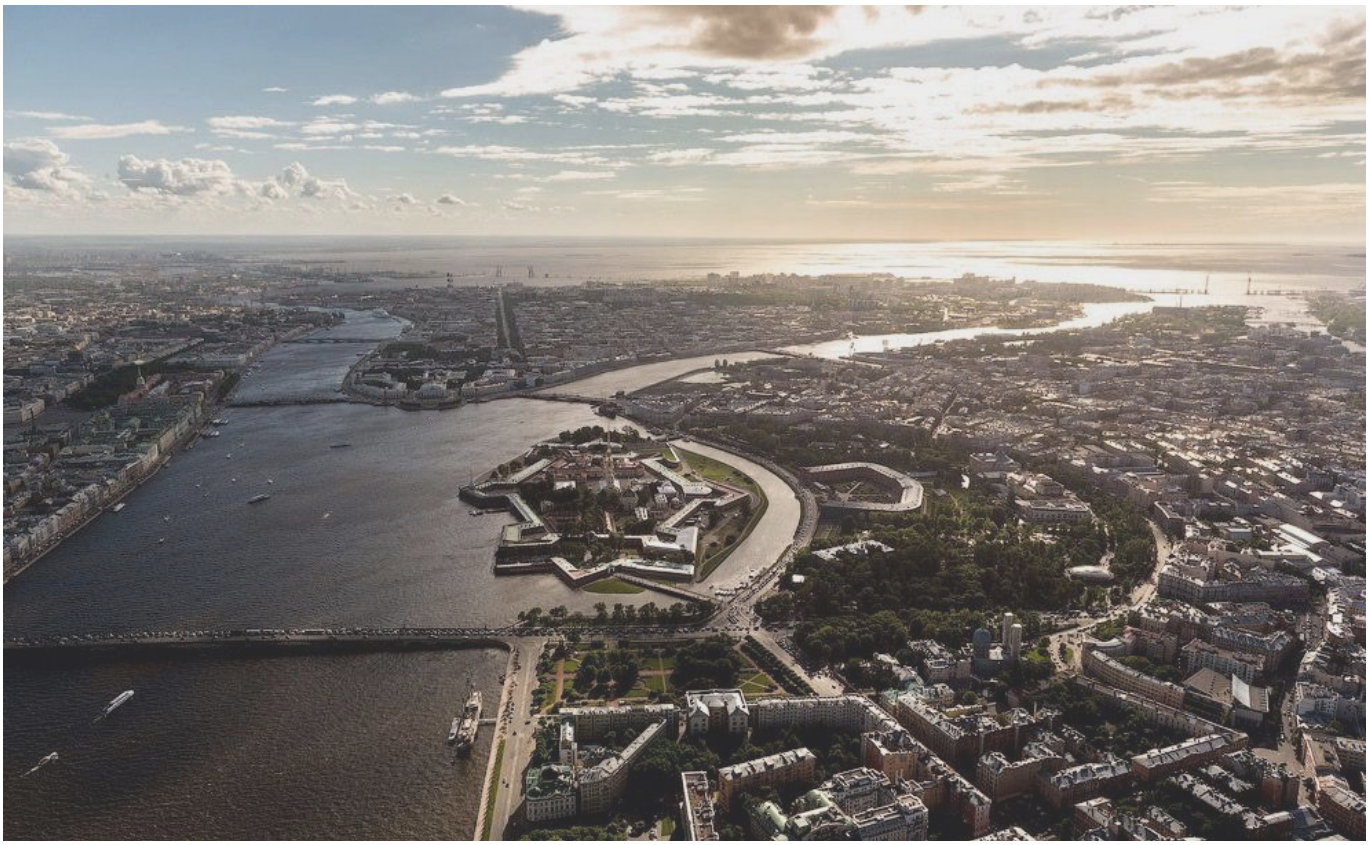
J3211, Сидельникова Дарья, 476010

ПРОЕКТИРОВАНИЕ БД

Тема 16: Поиск/покупка/аренда жилья

Лабораторная 6

(На примере https://cian.ru/)



Санкт-Петербург
2025 г

часть 1. партиционирование

Я выбрала таблицу listings (хранение объявлений о продаже и аренде недвижимости)
Потому что
- данные имеют время
- присутствует автогенерящийся идентификатор
Поле партиционирования created_at
Стратегия партиционирования range

Создание партиционированной таблицы

создание родительской таблицы

```
create table listings_new (
    id bigint not null,
    property_id bigint not null,
    deal_type_id smallint not null,
    price numeric(12,2) not null,
    deposit numeric(12,2),
    commission numeric(12,2),
    created_at timestamptz not null,
    updated_at timestamptz not null,
    is_active boolean not null
) partition by range (created_at);
```

создание партиций

```
create table listings_2025_01
partition of listings_new
for values from ('2025-01-01') to ('2025-02-01');

create table listings_2025_02
partition of listings_new
for values from ('2025-02-01') to ('2025-03-01');

create table listings_2025_03
partition of listings_new
for values from ('2025-03-01') to ('2025-04-01');
```

создание минимального индекса

```
create index on listings_new (id, created_at)
```

Настройка копирования данных (триггер)

```
create or replace function listings_insert()
returns trigger
language plpgsql
as $$
begin
    insert into listings_new (
        id,
        property_id,
        deal_type_id,
        price,
        deposit,
        commission,
        created_at,
        updated_at,
        is_active
    )
    values (
        new.id,
        new.property_id,
        new.deal_type_id,
        new.price,
        new.deposit,
        new.commission,
        new.created_at,
        new.updated_at,
        new.is_active
    );

    return new;
end;
$$;
```

триггер

```
create trigger trg_listings_mirror
after insert on listings
for each row
execute function listings_mirror_insert();
```

проверка работы триггера

```sql
select count(*) from realty.listings;select count(*) from
realty.listings_new;
```

Оба изменились одинаково

Перенос исторических данных

```sql
insert into realty.listings_new
select
    listing_id,
    property_id,
    deal_type_id,
    price,
    deposit,
    commission,
    created_at,
    updated_at,
    is_active
from realty.listings;
```

восстановление индексов и ограничений

```sql
create index concurrently idx_listings_2025_01_active_date
on realty.listings_2025_01 (deal_type_id, is_active, created_at);

create index concurrently idx_listings_2025_02_active_date
on realty.listings_2025_02 (deal_type_id, is_active, created_at);

create index concurrently idx_listings_2025_03_active_date
on realty.listings_2025_03 (deal_type_id, is_active, created_at);

create index concurrently idx_listings_2025_11_active_date
on realty.listings_2025_11 (deal_type_id, is_active, created_at);

create index concurrently idx_listings_2025_12_active_date
on realty.listings_2025_12 (deal_type_id, is_active, created_at);
```

```sql
alter table realty.listings_new
add constraint price_positive
check (price >= 0) not valid;

alter table realty.listings_new
validate constraint price_positive;
```

Проверка идентичности данных

```sql
select count(*) from realty.listings;
select count(*) from realty.listings_new;

select sum(listing_id) from realty.listings;
select sum(id) from realty.listings_new;
```

данные в 4 таблицах идентичны.

Results 1 | Results 1 (2) | Results 1 (3) | Results 1 (4) | Statistics 1

select sum(id) from realty.listings_n | Enter a SQL expression to filter results (use Ctrl+Spac

```
sum|
---+
 34|
```

Перенос нагрузки на новую таблицу

определение sequence

```sql
select
    seq.relname
from pg_class seq
join pg_sequence ps on seq.oid = ps.seqrelid
join pg_depend d on d.objid = seq.oid
join pg_class tbl on d.refobjid = tbl.oid
where tbl.relname = 'listings';
```

```
relname                |
-----------------------+
listings_listing_id_seq|
```

переключение таблиц

```
begin;

lock table realty.listings in access exclusive mode;
lock table realty.listings_new in access exclusive mode;

alter table realty.listings_new
alter column id set default nextval('listings_listing_id_seq');

alter sequence listings_listing_id_seq
owned by realty.listings_new.id;

alter table realty.listings rename to listings_old;
alter table realty.listings_new rename to listings;

commit;
```

Финальная проверка и удаление старой таблицы

```
insert into realty.listings (
    property_id,
    deal_type_id,
    price,
    deposit,
    commission,
    created_at,
    updated_at,
    is_active
)
values (
    1, 1, 444444, 444444, 44444,
    now(), now(), true
);

select *
from realty.listings
order by created_at desc
limit 5;

drop table realty.listings_old;
drop function realty.listings_mirror_insert;
```

listing_id	property_id	deal_type_id	price	deposit	commission	created_at	updated_at	is_active
13	1	1	444444.00	444444.00	44444.00	2025-12-21 17:23:44.913 +0300	2025-12-21 17:23:44.913 +0300	true
12	1	1	97000.00	97000.00	9700.00	2025-12-21 15:59:32.842 +0300	2025-12-21 15:59:32.842 +0300	true
4	1	1	45000.00	90000.00	5000.00	2025-11-07 13:29:10.431 +0300	2025-11-07 13:29:10.431 +0300	true
2	2	2	8500000.00		100000.00	2025-11-07 13:16:28.223 +0300	2025-11-07 13:16:28.223 +0300	true
3	3	1	120000.00	240000.00	15000.00	2025-11-07 13:16:28.223 +0300	2025-11-07 13:16:28.223 +0300	true

перенос завершён успешно, новая таблица используется

Выводы по проблемам параллелизма и способам их решения

Проблемы параллельной работы с бд:
- конкурентные вставки и обновления данных
- блокировки таблиц при изменении структуры
- потеря или дублирование данных
- ошибки из-за одновременного доступа к одним и тем же строкам

Способы решения проблем параллелизма:
- использование триггера для дублирования новых данных в обе таблицы
- выполнение критических операций (переименование таблиц) внутри транзакции
- применение явных блокировок (access exclusive) только на короткое время
- создание индексов с учётом ограничений PostgreSQL, чтобы избежать долгих блокировок
- поэтапное выполнение операций и контроль состояния транзакций

Ответы на вопросы:

Что такое партиция с точки зрения postgresql?

Партиция — это физическая таблица, которая хранит часть данных основной партиционированной таблицы по заданному правилу.

Чем полезны партиции? какой в них смысл?

Партиции позволяют ускорять запросы за счёт работы только с нужной частью данных, упростить управление большими таблицами, быстро удалять или архивировать старые данные, снизить нагрузку на систему

Что лучше: периодически удалять данные или дропать ненужные партиции? почему?

Лучше дропать ненужные партиции, потому что операция выполняется значительно быстрее, не нагружает систему множественными delete и не блокирует таблицу надолго